

Customer Review Sentiment Analysis using NLP (Detailed Documentation)

1. Project Overview

This project builds an **NLP-based machine learning model** that classifies customer reviews into:

- Positive 😊
- Negative 😞
- Neutral 😐

The system processes raw text, converts it into numerical form, and predicts sentiment using a trained model.

2. How the Model Works (High-Level Flow)

User Input (Review Text)

↓

Text Preprocessing

↓

TF-IDF Vectorization

↓

Machine Learning Model (Logistic Regression)

↓

Prediction Output (Positive / Negative / Neutral)

3. Detailed Workflow (Step-by-Step)

◆ Step 1: Input Data

- Source: reviews.csv
- Format:

Review → "This product is amazing"

Label → Positive

]

◆ Step 2: Text Preprocessing

Raw Text:

"This product is AMAZING!!!"

After Processing:

"this product is amazing"

Operations performed:

- Lowercasing
- Removing punctuation
- Removing unnecessary spaces
- (Optional) Stopword removal

◆ Step 3: Feature Engineering (TF-IDF)

Text is converted into numbers using **TF-IDF**:

Text → Numerical Vector

"good product" → [0.2, 0.8, 0.0, 0.5]

TF-IDF gives importance to:

- Important words → Higher weight
- Common words → Lower weight

◆ Step 4: Train-Test Split

Dataset → Split into:

- Training Data (80%)
- Testing Data (20%)

Purpose:

- Train model on known data
- Test performance on unseen data

◆ Step 5: Model Training

Algorithm used:

👉 **Logistic Regression**

Model learns:

Word Patterns → Sentiment

Example:

- "amazing", "excellent" → Positive
- "worst", "bad" → Negative

◆ Step 6: Prediction

Input:

"This product is very good"

Model Output:

Positive

◆ Step 7: Evaluation

Accuracy = Correct Predictions / Total Predictions

Example:

Accuracy: 85%

4. System Architecture Diagram

reviews.csv



Data Loading



Text Cleaning



TF-IDF Vectorizer



Train/Test Split



Logistic Regression
Model Training



Prediction Output
(Sentiment)

5. Model Logic (Behind the Scenes)

The model works by:

1. Learning word importance using TF-IDF
2. Identifying patterns between words and sentiments
3. Assigning probability scores for each class

Example:

Input: "Worst product ever"

Model Probabilities:

Positive → 0.05

Neutral → 0.10

Negative → 0.85

Final Output → Negative

6. Input–Output Flow

Input:

"This product is not good"

Processing:

Text → Cleaned → Vectorized → Model

Output:

Negative 😞

7. Performance Metrics

- Accuracy
- Precision
- Recall

- F1 Score

Example:

Accuracy: 85%

Precision: 84%

Recall: 83%

F1 Score: 83.5%

8. Recruiter-Focused Highlights

- ✓ End-to-end ML pipeline
- ✓ NLP preprocessing + feature engineering
- ✓ Real-world dataset simulation
- ✓ Model evaluation included
- ✓ Scalable architecture

9. Real-World Applications

- Amazon product reviews
- Twitter sentiment analysis
- Customer feedback systems
- Brand reputation monitoring

10. Limitations

- Cannot detect sarcasm
- Limited context understanding
- Depends on training data quality

11. Future Improvements

- Deep Learning models (LSTM, BERT)
- Real-time API deployment
- Multilingual sentiment detection

- Integration with dashboards

12. Conclusion

This project demonstrates a complete **NLP pipeline** from raw text to prediction. It effectively classifies sentiment using machine learning and can be extended for real-world applications.